Top-k

Pregunta 1
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Un modelo de lenguaje genera palabra por palabra. En el texto se miden dos momentos distintos. En uno, el contexto es restrictivo y hay pocas continuaciones razonables: top-p = 0,9 conserva 188 palabras candidatas. En el otro, el contexto admite muchísimas continuaciones: conserva 654. Top-k = 10 conserva 10 candidatas en los dos momentos. ¿Qué muestra esa medición?

Seleccione una:

a. Que top-p es más lento porque tiene que ordenar el vocabulario entero en cada paso, y top-k solo busca diez.
b. Que la temperatura y top-p hacen exactamente lo mismo sobre la distribución, y conviene usar uno solo de los dos.
c. Que top-p se adapta: con distribución repartida conserva muchas palabras, con distribución concentrada pocas; top-k siempre la misma cantidad. ✓ Sí. Donde el modelo duda entre cientos de palabras razonables, top-k = 10 excluye opciones perfectamente válidas; donde está seguro, deja entrar candidatas malas. Por eso top-p es lo que exponen las APIs, combinado con la temperatura.
d. Que top-p con p = 0,9 es más permisivo que top-k = 10 y por eso genera texto más aleatorio siempre.

Resolución. Top-k conserva las k más probables, esté el modelo seguro o indeciso. Top-p conserva las más probables hasta acumular p: la cantidad sigue a la indecisión del modelo (188 candidatas en un momento, 654 en el otro). La temperatura, en cambio, deforma la distribución antes de truncar.

La respuesta correcta es: Que top-p se adapta: con distribución repartida conserva muchas palabras, con distribución concentrada pocas; top-k siempre la misma cantidad.

5003

Pregunta 2
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

El vocabulario de los cuentos tiene **5.003** tokens. ¿Qué perplejidad tiene sobre validación un modelo que reparte la probabilidad por igual entre todas las palabras? Respondé un entero.

Respuesta: 5003 ✓

Perplejidad = exp(entropía cruzada media) = exp(ln 5003) = 5.003: «duda entre 5.003 palabras». Y es lo que da la LSTM sin entrenar (4.993), la verificación de que la medida está bien conectada.

Resolución. Perplejidad = $\exp(-(1/N)\,\Sigma \ln P(w_i \mid \text{contexto}))$. Con P = 1/V en todas las posiciones: exp(ln V) = **V** = **5.003**. Lectura: entre cuántas palabras duda el modelo. Unigrama 433, bigrama interpolado 121, trigrama interpolado 101, LSTM 73, modelo perfecto 1.

La respuesta correcta es: 5003

cuentos

Pregunta 3
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Para entrenar el clasificador de reseñas hizo falta un corpus donde alguien hubiera puesto, reseña por reseña, la cantidad de estrellas. Para entrenar un modelo de lenguaje alcanza con texto suelto: cuentos, noticias, páginas web, sin que nadie los anote. ¿De dónde salen las etiquetas del modelo de lenguaje, y por qué esa diferencia importa?

Seleccione una:

a. El modelo de lenguaje se entrena sin función de pérdida, porque no hay etiquetas contra las que comparar.
b. No hay etiquetas: la capa de salida tiene una unidad por palabra del vocabulario y el modelo se entrena por reconstrucción, sin comparar contra nada.
c. El modelo de lenguaje usa una LSTM y el clasificador un promedio; la recurrencia es lo que cambia todo.
d. Que los datos salen del texto mismo corrido un lugar: nadie etiqueta nada, y por eso escala a cantidades enormes de texto. ✓ Sí. Cada oración de 20 tokens produce 19 preguntas con respuesta. Es la razón por la que los modelos de lenguaje pudieron crecer como crecieron.

Resolución. Las trae el texto: entrada, un fragmento; objetivo, el mismo fragmento corrido un lugar. El texto trae la respuesta adentro (autosupervisado). Todo lo demás —logits, softmax, entropía cruzada— es lo mismo que clasificar; lo que representa cada clase es una palabra en vez de una cantidad de estrellas.

La respuesta correcta es: Que los datos salen del texto mismo corrido un lugar: nadie etiqueta nada, y por eso escala a cantidades enormes de texto.

encoder-decoder

Pregunta 4
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

En un traductor encoder-decoder sin atención, lo único que el decoder recibe del encoder es su estado final. ¿Cuál es el problema de ese diseño, el que la atención viene a resolver?

Seleccione una:

- a. Toda la oración de entrada tiene que caber en un único vector, el estado final del encoder; con entradas largas se pierde detalle. ✓ Sí. Cada estado oculto del encoder contiene información útil (qué palabra se corresponde con cuál, a qué sustantivo refiere un artículo) y todo eso se pierde si el único enlace es el estado final. Que invertir la oración de entrada mejore el resultado ya era una señal de que el problema estaba ahí.
- b. El decoder no puede generar oraciones más largas que la entrada, porque produce una palabra por cada estado del encoder.
- c. El encoder y el decoder tienen vocabularios distintos y el estado oculto no sabe traducir entre índices.
- d. La LSTM del encoder olvida el principio de la oración; con una GRU el problema desaparece.

Resolución. El único enlace entre las dos mitades es el estado final (h, c): un vector de tamaño fijo para cualquier largo de oración. La atención lleva a cada paso del decoder un **contexto**, suma ponderada de *todos* los estados del encoder, con pesos distintos para cada palabra que se está produciendo. En el notebook: BLEU 26,2 → 30,7 con atención, y la ganancia se ve en todos los largos.

La respuesta correcta es: Toda la oración de entrada tiene que caber en un único vector, el estado final del encoder; con entradas largas se pierde detalle.

LSTM

Pregunta 5
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Un clasificador recurrente lee reseñas de **48 palabras** y decide la clase mirando únicamente el estado oculto de la última posición, que es donde se calcula la pérdida. Para que la **primera** palabra de la reseña influya en algo, su gradiente tiene que viajar hacia atrás por los 48 pasos de la recurrencia.

Se mide, con el mismo experimento sobre las dos arquitecturas, cuánto gradiente le llega al vector (el *embedding*) de cada posición. En la posición 0, la RNN simple recibe un gradiente **ocho órdenes de magnitud menor** que la LSTM: un 1 seguido de ocho ceros de diferencia. ¿Qué implica?

Seleccione una:

- a. Que la LSTM elimina el desvanecimiento por completo gracias al estado de celda, y puede recordar secuencias de cualquier largo.
- b. Que en la RNN simple la primera palabra de la reseña no participa del aprendizaje; la LSTM mitiga el desvanecimiento con sus compuertas, aunque no lo elimina. ✓ Sí. Todas las curvas bajan (con 48 pasos el desvanecimiento es inevitable), pero la distancia entre ellas es de varios órdenes de magnitud. Y la LSTM también decae, solo que mucho más lento.
- c. Que la RNN simple tiene gradientes que explotan al retropropagar por 48 pasos, y por eso se usa recorte de gradientes.
- d. Que la RNN simple está mal inicializada; con pesos más grandes el gradiente llegaría igual que en la LSTM.

Resolución. Gradientes que se desvanecen: la señal que llega a las primeras posiciones se pierde en el redondeo. Las compuertas de la LSTM (y de la GRU) mitigan el problema: el estado de celda solo se multiplica por la compuerta de olvido y se le suma lo nuevo, así que si la compuerta está cerca de 1 la información y el gradiente cruzan el paso casi intactos. Es una mitigación, no una cura.

La respuesta correcta es: Que en la RNN simple la primera palabra de la reseña no participa del aprendizaje; la LSTM mitiga el desvanecimiento con sus compuertas, aunque no lo elimina.

Paso

Pregunta 6
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Ordená los pasos para generar texto con el modelo ya entrenado, a partir de una semilla.

Paso 1: Alimentar la semilla al modelo y obtener los logits de la palabra siguiente ✓
Paso 2: Aplicar las perillas (temperatura, top-k o top-p) y sortear una palabra de la distribución ✓
Paso 3: Agregar la palabra elegida a la salida ✓
Paso 4: Realimentar esa palabra como próxima entrada, con el estado de la LSTM a cuestas ✓
Paso 5: Repetir hasta el largo pedido o hasta el token de fin ✓

Resolución. Semilla → logits → distribución (con perillas) → sorteo → agregar a la salida → realimentar con el estado → repetir. El modelo no «sabe» lo que dice: en cada paso resuelve la misma pregunta de siempre, y la coherencia del texto largo es un efecto emergente de repetirla.

Empareja

Pregunta 7
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Emparejá cada tarea con la forma de la red recurrente.

Etiquetar cada palabra de una oración con su categoría gramatical.	Muchos a muchos, alineado
Traducir una oración entera a otro idioma.	Muchos a muchos, diferido (encoder-decoder)
Describir una imagen con una oración.	Uno a muchos
Clasificar una reseña con una cantidad de estrellas.	Muchos a uno

Bien: clasificar es muchos a uno, describir una imagen uno a muchos, etiquetar palabras muchos a muchos alineado, y traducir necesita leer todo antes: diferido.

Resolución. Muchos a uno: secuencia → valor (clasificar). **Uno a muchos**: valor → secuencia (describir una imagen). **Muchos a muchos alineado**: una salida por entrada (etiquetar palabras). **Diferido**: hay que leer toda la entrada antes de producir, porque no hay correspondencia uno a uno ni el mismo orden (traducir): es el esquema encoder-decoder.

La respuesta correcta es: Etiquetar cada palabra de una oración con su categoría gramatical. → Muchos a muchos, alineado, Traducir una oración entera a otro idioma. → Muchos a muchos, diferido (encoder-decoder), Describir una imagen con una oración. → Uno a muchos, Clasificar una reseña con una cantidad de estrellas. → Muchos a uno

beam search

Pregunta 8
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

¿Por qué elegir la palabra más probable en cada paso no garantiza la traducción más probable, y qué hace beam search al respecto?

Seleccione una:

a. Porque el voraz solo mira la probabilidad de un token; beam search evalúa todas las secuencias posibles y elige la mejor.
b. Porque el voraz no usa el token `<eos>` y no sabe cuándo parar; beam search sí lo usa.
c. Porque el argmax es determinista y produce bucles; beam search sortea entre las k más probables para romperlos.
d. Porque un token muy probable ahora puede llevar a un camino peor después; beam search conserva k secuencias parciales y las extiende.

Sí. Es la diferencia entre tomar siempre la calle que parece más despejada en cada esquina y elegir el mejor recorrido completo. La búsqueda exhaustiva es inviable (crece exponencialmente), y beam es el punto medio.

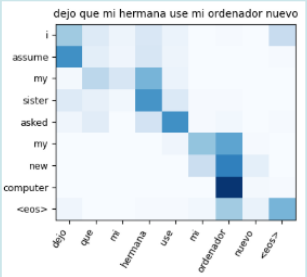
Resolución. La secuencia más probable no es la que elige el token más probable en cada paso. Beam search extiende k hipótesis en paralelo, se queda con las k mejores por probabilidad acumulada, y termina cuando emiten `<eos>`. Encuentra a menudo mejores traducciones que el voraz sin el costo de la búsqueda exhaustiva.

La respuesta correcta es: Porque un token muy probable ahora puede llevar a un camino peor después; beam search conserva k secuencias parciales y las extiende.

Pesos

Pregunta 9
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Un traductor español→inglés con atención deja ver, en cada paso, cuánto peso le dio a cada palabra de la entrada. El mapa de abajo es el de una oración de prueba: cada **fila** es una palabra que el modelo produjo, cada **columna** una palabra de la entrada, y cuanto más oscuro, más peso.



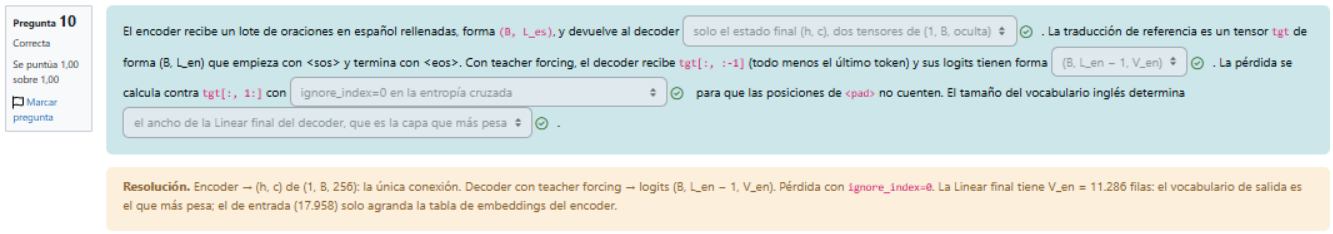
Los pesos se concentran sobre algo parecido a una diagonal, y el bloque final se corre respecto de ella, donde el inglés ordena distinto que el español. ¿Qué dice ese mapa sobre cómo se obtuvo esa correspondencia entre palabras?

Seleccione una:

a. Es un artefacto de la visualización: como los pesos de cada fila suman 1, siempre se concentran en alguna columna.
b. Se obtuvo porque el encoder lee la oración de entrada en el orden del idioma de salida, que es para lo que se invierte la secuencia.
c. Nadie la definió: los pesos de atención se aprenden junto con el resto del modelo, como efecto secundario de aprender a traducir.
d. Se obtuvo de un diccionario bilingüe con el que se inicializan los pesos de atención antes de entrenar.

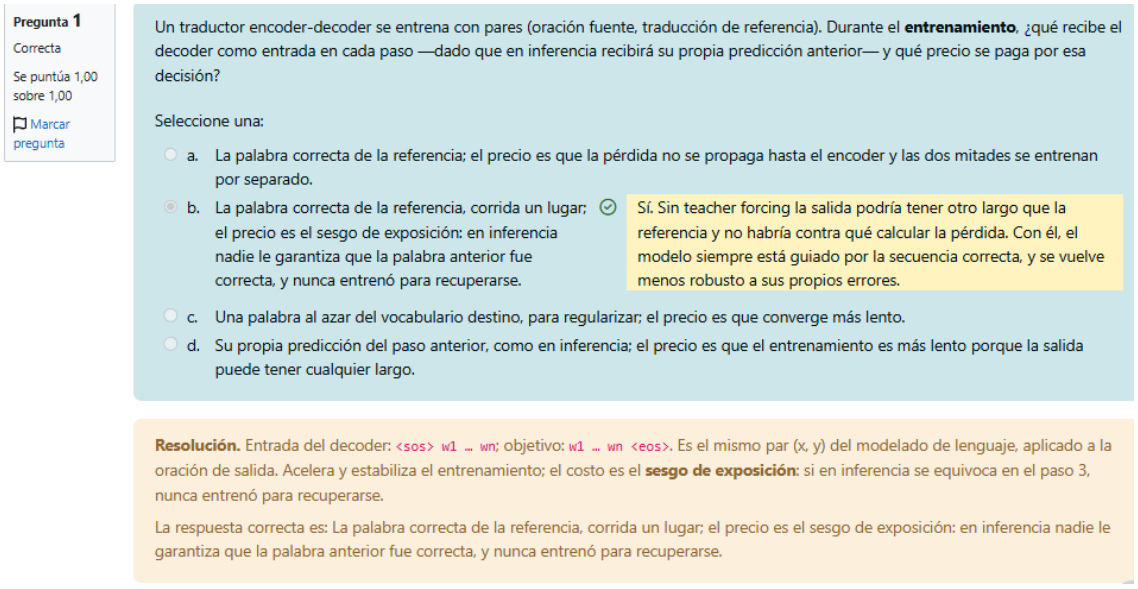
Sí. El módulo que calcula los pesos se entrena con la misma pérdida que el encoder y el decoder; nadie le enseñó qué palabra va con cuál. La alineación aparece sola, y de ahí el título del paper que introdujo la idea: *jointly learning to align and translate*.

Encoder-decoder

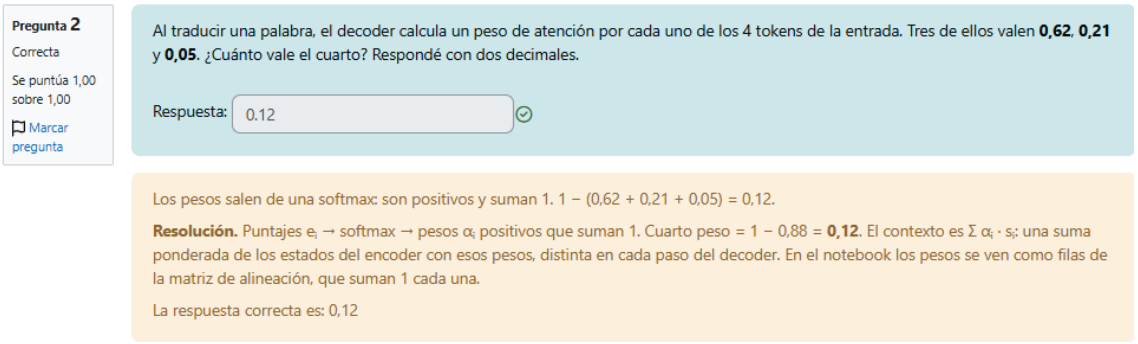


test 2

encoder-decoder



decoder



codigo

Pregunta 3
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Una capa recurrente simple, escrita a mano, sobre un lote `x` de forma `(B, T, entrada)`:

```
h = torch.zeros(B, oculta)
for t in range(T):
    x_t = x[:, t]                      # (B, entrada)
    h = torch.tanh(x_t @ W_ih.T + h @ W_hh.T + b)
estado_final = h                       # (B, oculta)
```

¿Qué recorre ese bucle y por qué eso define el costo de la arquitectura?

Seleccione una:

a. Recorre los ejemplos del lote, uno por vez, y dentro de cada uno las posiciones; por eso los lotes grandes son lentos.

b. Recorre las posiciones de la secuencia, con las B secuencias del lote juntas; el paso t necesita el t−1, así que no se paraleliza. ✓ Sí. Es la confusión más común: el for no recorre los ejemplos. Duplicar el largo duplica el tiempo, y no hay GPU que lo arregle; es una de las razones por las que aparecen los transformers.

c. Recorre las posiciones, pero PyTorch lo paraleliza al compilar `nn.RNN`, así que el largo no afecta el tiempo.

d. Recorre las unidades ocultas, calculando cada una con su fila de W_hh; el costo crece con el tamaño de la capa.

Resolución. El bucle es sobre el **tiempo**: `x[:, t]` es la palabra t de todas las secuencias del lote, y `h` guarda B estados. Cada paso depende del anterior, así que las posiciones no se pueden calcular en paralelo. Con `nn.LSTM` el for sigue existiendo, compilado.

La respuesta correcta es: Recorre las posiciones de la secuencia, con las B secuencias del lote juntas; el paso t necesita el t−1, así que no se paraleliza.

BLEU

Pregunta 4
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Contra las referencias «i am going to sleep» y «i have to go to sleep», la hipótesis «i am going to bed» obtiene un BLEU mucho más alto que «i am going to», aunque la segunda no tiene ninguna palabra incorrecta. ¿Por qué?

Seleccione una:

a. Porque BLEU premia la cantidad de palabras: cuanto más larga la hipótesis, más n-gramas pueden coincidir.

b. Porque «i am going to» termina sin punto y sacrebleu descarta las hipótesis mal formadas.

c. Porque BLEU multiplica la precisión de n-gramas por una penalización de brevedad: lo que falta también castiga. ✓ Sí. Sin la penalización, una traducción de una sola palabra correcta tendría precisión perfecta. BLEU cuenta n-gramas (1 a 4) que aparecen en alguna referencia, y castiga las hipótesis más cortas que la referencia.

d. Porque BLEU mide recall: cuántas palabras de la referencia aparecen en la hipótesis, y «bed» cuenta como «sleep» por sinonimia.

Resolución. BLEU = penalización de brevedad × media geométrica de las precisiones de 1- a 4-gramas contra el conjunto de referencias. «i am going to» tiene precisión perfecta pero es más corta que la referencia y la penalización la hunde; «i am going to bed» tiene un unigrama que no coincide, pero el largo correcto. Y se mide sobre el corpus, con varias referencias por oración.

La respuesta correcta es: Porque BLEU multiplica la precisión de n-gramas por una penalización de brevedad: lo que falta también castiga.

BAHDANAU

Pregunta 5
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Ordená lo que pasa en **un paso** del decoder con atención de Bahdanau, dado el estado del decoder del paso anterior y los estados del encoder.

Paso 1: Calcular un puntaje por estado del encoder comparándolo con el estado del decoder (v · tanh(W h_dec + U s_i)) ✓

Paso 2: Pasar los puntajes por una softmax: pesos positivos que suman 1 (con −∞ en las posiciones de relleno) ✓

Paso 3: Calcular el contexto como la suma ponderada de los estados del encoder con esos pesos ✓

Paso 4: Concatenar el contexto con el embedding de la palabra anterior y dar un paso de la LSTMCell ✓

Paso 5: Pasar el h nuevo por la capa de salida para obtener los logits de la palabra siguiente ✓

Resolución. Puntajes → softmax → contexto → [contexto ; emb(anterior)] → LSTMCell → logits. Como el contexto depende del estado del decoder del paso anterior, el forward tiene que ir paso a paso: por eso el notebook cambia `nn.LSTM` por `nn.LSTMCell`. Los pesos de cada paso forman una fila de la matriz de alineación.

ESTADO FINAL LSTM

Pregunta 6
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Un clasificador recurrente decide la clase a partir del **estado final** de la LSTM. Los lotes se arman rellenando cada reseña con <pad> hasta el largo de la más larga del lote, y el tensor rellenado entra tal cual a la capa recurrente, sin avisarle dónde termina cada reseña. ¿Qué problema tiene eso?

Seleccione una:

a. Que los pasos de relleno entran en el cálculo de la pérdida y el modelo termina aprendiendo a predecir `<pad>`.
b. Ninguno serio: como el embedding usa `padding_idx=0`, el vector de relleno es cero y el estado se queda quieto.
c. El único problema es de cómputo: se procesan pasos que no aportan nada y el entrenamiento tarda más.
d. La misma reseña da un estado final distinto según cuánto relleno le tocó: su predicción depende de con quién viajó en el lote. ✓

Exacto. La capa sigue actualizando el estado en cada paso de relleno, de modo que el estado final no es el de la última palabra sino el de varios pasos después. Es el caso de manual para `pack_padded_sequence`: se le pasan los largos reales y la capa se detiene donde termina el texto.

CAPA RECURRENTE SIMPLE

Pregunta 7
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Una capa recurrente simple (`nn.RNN`) tiene **2** features de entrada y **3** unidades. ¿Cuántos parámetros tiene en total?

Respuesta: 21 ✓

W_ih (3×2) = 6, b_ih = 3, W_hh (3×3) = 9, b_hh = 3: 21. Los 12 «de más» respecto de una capa lineal (9) son toda la memoria del modelo.

Resolución. W_ih: 3×2 = 6; b_ih: 3; W_hh: 3×3 = 9; b_hh: 3. Total **21**, contra 9 de una capa lineal de 2 a 3. La LSTM tiene cuatro juegos (84) y la GRU tres (63): es lo que muestra la tabla de costo del notebook (55.040 / 220.160 / 165.120 con DIM = 300 y OCULTA = 128).

La respuesta correcta es: 21

5003

Pregunta 8
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Un modelo de lenguaje de vocabulario 5.003 tiene **dos** tablas con una fila por palabra: la de **entrada**, que convierte cada índice en un vector, y la de **salida**, que convierte el estado de la red en un puntaje por palabra. Terminado el entrenamiento se buscan, por similitud de coseno, las filas más parecidas a la de «rojo». En la tabla de **salida** son *negro, verde, blanco, rubio*; en la de **entrada**, palabras sin relación entre sí. ¿Por qué aprendió español una tabla y la otra no?

Seleccione una:

a. Porque la tabla de salida se inicializa con vectores preentrenados de fastText y la de entrada con valores al azar.
b. Porque la tabla de entrada se congela después de las primeras épocas para estabilizar el entrenamiento.
c. Porque la tabla de salida es más grande (256 columnas contra 128) y tiene más capacidad para representar semántica.
d. Porque la fila de salida de cada palabra recibe gradiente en todos los pasos (la softmax empuja a todas), y la de entrada solo cuando la palabra aparece. ✓

Sí: en un lote, 718 filas de la tabla de entrada reciben gradiente contra las 5.003 de la de salida. Y es señal comparativa: «rojo» y «verde» reciben gradientes parecidos ante estados parecidos, y eso las acerca.

Resolución. Medido en un lote: 718 de 5.003 filas de entrada reciben gradiente; 5.003 de 5.003 de salida. La fila de salida de cada palabra se compara con todas las demás en cada paso. Los modelos actuales atan las dos tablas (*weight tying*), que es el tema del laboratorio.

La respuesta correcta es: Porque la fila de salida de cada palabra recibe gradiente en todos los pasos (la softmax empuja a todas), y la de entrada solo cuando la palabra aparece.

Muchos a muchos diferido

Pregunta 9
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

El modelo secuencia a secuencia de Sutskever y otros (2014) se describe como **muchos a muchos diferido**, para distinguirlo del muchos a muchos *alineado* (una salida por cada entrada, en el mismo orden). ¿Qué significa «diferido» y qué problema de la traducción resuelve?

Seleccione una:

- a. Que no empieza a producir salida hasta haber leído toda la entrada: eso le permite reordenar, condensar o expandir según lo pida el idioma destino. ✓ Sí. «I like almond milk» y «Mi piace il latte di mandorla» no coinciden ni en cantidad ni en orden de palabras; traducir palabra por palabra a medida que se lee queda descartado de entrada.
- b. Que el decoder espera a que el encoder termine de entrenarse antes de empezar su propio entrenamiento.
- c. Que la salida se calcula un paso después de la entrada, como en el etiquetado gramatical, para poder mirar una palabra de contexto hacia adelante.
- d. Que la traducción se genera al revés, de la última palabra a la primera, para acercar el final de la entrada al comienzo de la salida.

Resolución. Diferido = leer toda la entrada, comprimirla en un estado, y recién entonces producir. El encoder codifica la oración en una representación oculta de tamaño fijo; el decoder la decodifica hacia la traducción. La demora deliberada es lo que permite que la salida tenga otro largo y otro orden.

La respuesta correcta es: Que no empieza a producir salida hasta haber leído toda la entrada: eso le permite reordenar, condensar o expandir según lo pida el idioma destino.

Decoder Encoder BAHDANAU

Pregunta 10
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

En la atención de Bahdanau, el puntaje de cada posición de la entrada sale de comparar el **estado del decoder** con **el estado del encoder** de esa posición, y el contexto es la suma ponderada de **esos mismos estados del encoder**. La formulación general habla en cambio de tres papeles: **consulta** (lo que se busca), **clave** (contra qué se compara) y **valor** (qué se devuelve). ¿Qué relación hay entre las dos?

Seleccione una:

- a. Bahdanau es un caso particular: clave y valor eran el mismo estado del encoder y la consulta el estado del decoder. ✓ Sí. Con un vector para todo (predecir el token, servir de clave, aportar contenido) el modelo es más difícil de entrenar. Con W_q, W_k, W_v cada uno se especializa: es el mecanismo de BERT y GPT.
- b. La consulta la produce el encoder y las claves el decoder; en Bahdanau era al revés.
- c. Consulta, clave y valor son tres cabezas de atención que se calculan en paralelo y se concatenan.
- d. Es un mecanismo distinto: Bahdanau compara con un MLP y la versión general con un buscador externo que devuelve el valor más parecido.

Resolución. Pesos = softmax(consulta · claves / √d); contexto = Σ pesos · valores. Consulta: del decoder; claves y valores: del encoder, con proyecciones distintas. Bahdanau: clave = valor = estado del encoder, consulta = estado del decoder. En el notebook, Q/K/V y Bahdanau empatan en BLEU (31,9 vs 30,7, dentro del ruido) pero los mapas de atención cambian.

La respuesta correcta es: Bahdanau es un caso particular: clave y valor eran el mismo estado del encoder y la consulta el estado del decoder.

test 3

Clasificador de reseñas

Pregunta 1
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Un clasificador de reseñas pasa un lote de **B** reseñas de **L** tokens (ya empaquetado con `pack_padded_sequence`, que le avisa a la capa dónde termina cada reseña) por una LSTM de **una** capa y **128** unidades:

```
salidas, (h, c) = lstm(entrada_empaquetada)
```

`salidas` tiene un vector por [posición de la secuencia] ✓, `h` tiene forma [(1, B, 128): una fila por capa] ✓, y para clasificar la reseña se usa [h[-1]: el estado después de la última palabra real] ✓. El tensor `c` es [la memoria interna de la celda, que viaja de paso en paso y no aparece en salidas] ✓.

Resolución. `salidas` (B, L, oculta): el h de cada posición (el modelo de lenguaje las va a usar todas). `h`, `c`: (capas, B, oculta), el estado final; por eso `h[-1]`. Con `pack_padded_sequence`, h[-1] corresponde a la última palabra real. `c` es la memoria interna de la LSTM; RNN y GRU devuelven solo h.

traductor encoder-decoder

Pregunta 2
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Un traductor encoder-decoder ya entrenado traduce un lote de B oraciones eligiendo en cada paso la palabra más probable. Ordená los pasos.

Paso 1: Codificar: el encoder lee cada oración entera y deja su resumen en el estado (h, c)
Paso 2: Arrancar: la primera entrada del decoder es para cada oración del lote
Paso 3: Un paso del decoder: recibe la palabra anterior embebida y el estado, y devuelve h y el estado nuevo
Paso 4: Elegir: la capa de salida da un puntaje por palabra y argmax se queda con la más alta
Paso 5: Guardar la palabra elegida y volver al paso del decoder con ella como entrada
Paso 6: Apilar: los max_len tensores (B, 1) se vuelven un tensor (B, max_len), y se corta en el primer

Resolución. Codificar → arrancar con `<sos>` → paso del decoder → argmax → guardar y realimentar → apilar y cortar en `<eos>`. Los pasos 3 a 5 se repiten `max_len` veces para las B oraciones a la vez; alimentarse con la propia predicción es lo que en entrenamiento se reemplaza por teacher forcing.

entropía cruzada media

Pregunta 3
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

El modelo entrenado tiene una entropía cruzada media de **4,29** sobre validación. ¿Cuál es su perplejidad? Respondé un entero.

Respuesta: 73

exp(4,29) ≈ 73: el modelo duda, en promedio, entre 73 palabras. Contra 101 del mejor trigrama y 5.003 del uniforme.

Resolución. Perplejidad = e^4,29 ≈ **73**. Es la exponencial de la entropía cruzada de siempre: convierte «nats de sorpresa» en «entre cuántas palabras duda». Y va en las dos direcciones: e^(−4,29) ≈ 0,0137 es la probabilidad media que le asigna a la palabra correcta, o sea 1/73.

La respuesta correcta es: 73

traductor sin atención

Pregunta 4
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

El traductor sin atención hace el decoder entero con una llamada a `nn.LSTM` sobre la secuencia completa. El traductor con atención usa `nn.LSTMCell` y un bucle paso a paso. ¿Por qué?

Seleccione una:

a. Porque la entrada de cada paso incluye el contexto, que depende del estado del decoder anterior: no se puede armar la secuencia de antemano.
Sí. nn.LSTM recibe todas las entradas juntas y hace el bucle por dentro; con atención, la entrada del paso t recién se conoce cuando terminó el paso t−1. Las dos clases comparten los pesos: en el notebook se copian de una a otra y dan lo mismo.

b. Porque `nn.LSTMCell` es más rápida que `nn.LSTM` cuando las secuencias son cortas, como las del decoder.

c. Porque `nn.LSTM` no acepta un estado inicial y el decoder tiene que arrancar desde el estado del encoder.

d. Porque con atención el decoder es bidireccional y `nn.LSTM` solo puede procesar la secuencia en un sentido.

Resolución. `nn.LSTM` = la misma celda aplicada a toda la secuencia, con el bucle adentro. `nn.LSTMCell` = un paso. Con atención, la entrada del paso t es [contexto_t ; emb(w_{t−1})], y contexto_t se calcula con h_{t−1}: hay que ir paso a paso. Mismos pesos, mismo cálculo; distinto quién escribe el bucle.

La respuesta correcta es: Porque la entrada de cada paso incluye el contexto, que depende del estado del decoder anterior: no se puede armar la secuencia de antemano.

estrategia voraz

Pregunta 5
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Generando con la estrategia voraz (siempre la palabra más probable), tanto el bigrama por conteo como la LSTM terminan repitiendo la misma frase para siempre. ¿Por qué?

Seleccione una:

- a. Porque una regla determinista que vuelve a un estado ya visitado repite el mismo camino; la LSTM tarda más en caer, pero cae. ✓ Sí. Y la LSTM cae en el mismo bucle desde tres semillas distintas, porque «no se lo había dicho» es un atractor: desde muchos estados es la continuación más probable. La salida es sortear en vez de elegir el máximo.
- b. Porque el vocabulario es chico y las palabras más probables se agotan rápido, así que se vuelve a las mismas.
- c. Porque la softmax concentra casi toda la probabilidad en una palabra y las demás quedan en cero.
- d. Porque el modelo está sobreajustado y memorizó una frase del corpus; con más regularización no pasaría.

Resolución. Elegir siempre la más probable es determinista: si el proceso vuelve a un estado ya visitado, repite el camino para siempre. En el notebook la primera repetición de un 4-grama aparece entre los tokens 7 y 23. Muestrear de la softmax rompe el bucle, y temperatura, top-k y top-p regulan cuánto azar se admite.

La respuesta correcta es: Porque una regla determinista que vuelve a un estado ya visitado repite el mismo camino; la LSTM tarda más en caer, pero cae.

atención Q/K/V

Pregunta 6
Incorrecta
Se puntúa 0,00 sobre 1,00
Marcar pregunta

Una atención Q/K/V trabaja con vectores de **256** dimensiones: proyecta consulta, clave y valor con tres matrices y calcula un único reparto de pesos sobre la entrada. Se la reemplaza por una atención de **cuatro cabezas de 64**. ¿Qué cambia?

Seleccione una:

- a. Las cuatro cabezas calculan los mismos pesos y se promedian, lo que reduce el ruido de la atención. ✗ Cada cabeza tiene su propio reparto, y en el notebook son distintos entre sí (distancia media 0,65 entre pares). No se promedian: se concatenan los contextos.
- b. Los mismos vectores de 256 se leen de a 64: cada cabeza calcula su propio reparto y su contexto, y los cuatro se concatenan.
- c. Cada cabeza tiene sus propias W_q, W_k, W_v de 256 × 256, así que el modelo tiene cuatro veces más parámetros y por eso traduce mejor.
- d. Se concatenan los cuatro vectores de pesos (4 × L) y se pasan por una softmax conjunta antes de calcular un único contexto.

Resolución. Multi-cabeza: los 256 números de q, k, v se leen de a 64; cuatro productos punto, cuatro softmax, cuatro sumas ponderadas de 64; los cuatro contextos pegados dan 256. Parámetros: los mismos 3 × 256 × 256. Cada cabeza puede especializarse en un aspecto distinto. Se comparan los pesos de cada cabeza con la cabeza simple; los contextos se concatenan, los pesos no.

La respuesta correcta es: Los mismos vectores de 256 se leen de a 64: cada cabeza calcula su propio reparto y su contexto, y los cuatro se concatenan.

Pregunta 7
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

En el corpus de pares español→inglés, una misma oración en español aparece con varias traducciones. Si los pares se reparten al azar entre entrenamiento y validación, ¿qué pasa?

Seleccione una:

- a. La validación queda con más oraciones cortas, porque las oraciones con muchas traducciones son las más simples.
- b. Una oración fuente puede quedar en los dos lados con distintas traducciones: el modelo «valida» sobre oraciones que ya vio. ✓ Sí. En el notebook, el modelo con split ingenuo saca varios puntos más de BLEU sobre las fuentes que ya vio, y su pérdida de validación es mucho más baja, sin traducir mejor. La unidad que se reparte tiene que ser la oración fuente con todas sus traducciones.
- c. Nada grave: los pares son ejemplos independientes y el reparto al azar es la práctica estándar.
- d. Las traducciones múltiples confunden al modelo, que no sabe cuál aprender, y la pérdida no baja.

Resolución. Se separan 3.000 oraciones fuente para prueba (con todas sus traducciones como referencias) y 3.000 para validación; los pares de entrenamiento salen del resto. Con el split ingenuo, el 0 % de validación está en entrenamiento para el modelo correcto, pero casi todo para el ingenuo: por eso su pérdida «de validación» da más baja. Mismo conjunto, distinto lo que vio cada modelo.

La respuesta correcta es: Una oración fuente puede quedar en los dos lados con distintas traducciones: el modelo «valida» sobre oraciones que ya vio.

Pregunta 1

Sin contestar

Se puntúa como 0 sobre 1,00

Marcar pregunta

El encoder recibe un lote de oraciones en español rellenadas, forma `(B, L_es)`, y devuelve al decoder [ ] . La traducción de referencia es un tensor `tgt` de forma (B, L_en) que empieza con <sos> y termina con <eos>. Con teacher forcing, el decoder recibe `tgt[:, :-1]` (todo menos el último token) y sus logits tienen forma [ ] . La pérdida se calcula contra `tgt[:, 1:]` con [ ] para que las posiciones de `<pad>` no cuenten. El tamaño del vocabulario inglés determina [ ] .

Resolución. Encoder → (h, c) de (1, B, 256): la única conexión. Decoder con teacher forcing → logits (B, L_en − 1, V_en). Pérdida con `ignore_index=0`. La Linear final tiene V_en = 11.286 filas: el vocabulario de salida es el que más pesa; el de entrada (17.958) solo agranda la tabla de embeddings del encoder.